Einführung

Für die grafische Benutzeroberfläche unseres Projekts verwenden wir **JavaFX**, das moderne GUI-Toolkit von Java. JavaFX ermöglicht es uns, ansprechende Benutzeroberflächen mit minimalem Aufwand zu erstellen. Es unterstützt Komponenten wie Buttons, Slider, Menüs sowie 2D- und 3D-Grafiken, was es zur idealen Wahl für unser Projekt macht. Zusammen mit jBox2D bildet JavaFX eine zentrale Grundlage für die Darstellung und Interaktion im Spiel *Crazy Machines*.
JavaFX ersetzt das ältere Swing-Toolkit und bietet Vorteile wie deklaratives UI-Design mit FXML, eine hardwarebeschleunigte Rendering-Pipeline und eine gute Trennung von Logik und Darstellung.
Eine ausführliche und gut strukturierte Dokumentation zu JavaFX ist unter folgendem Link verfügbar: `https://fxdocs.github.io/docs/html5/`

Code-Snippet

Um die grundlegenden Funktionalitäten von JavaFX zu testen, wurde ein Beispielprogramm erstellt, welches ein einfaches Fenster mit einem farbigen Kreis und einer Schaltfläche darstellt. Beim Klick auf die Schaltfläche ändert sich der Text und der Kreis wechselt seine Farbe.
Die Datei befindet sich unter `src/main/java/mm/gui/SanityTest.java`.

```java
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import javafx.stage.Stage;

public class SanityTest extends Application {
    @Override
    public void start(Stage primaryStage) {
        // Kreis erstellen
        Circle circle = new Circle(75, Color.RED);

        // Button erstellen
        Button btn = new Button("Klick mich!");
        btn.setOnAction(e -> {
            btn.setText("Geklickt!");
            circle.setFill(Color.BLUE);
        });

        // Layout
        StackPane root = new StackPane();
        root.getChildren().addAll(circle, btn);

        Scene scene = new Scene(root, 300, 250);

        primaryStage.setTitle("Sanity Check: JavaFX");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Listing 1: JavaFX-Beispiel mit interaktiver Grafik

Im Beispiel wird ein JavaFX-Fenster erzeugt, das einen farbigen Kreis und eine Schaltfläche enthält. Die Methode `start` ist der Einstiegspunkt für jede JavaFX-Anwendung. Die Szene bildet das UI-Fundament, auf dem Komponenten wie der `StackPane` zentral angeordnet werden.
Durch `btn.setOnAction(...)` wird ein Event-Handler registriert, der sowohl den Text des Buttons als auch die Farbe des Kreises verändert. Damit wird sowohl die Interaktivität als auch das Zeichnen einfacher 2D-Formen demonstriert.

Hürden und Use-Case

JavaFX ist gut dokumentiert, jedoch unterscheidet sich sein Architekturmodell erheblich von klassischen Java-Anwendungen. Besonders die Trennung in `FXML`, Controller-Klassen und Anwendungsklassen kann anfangs verwirrend sein. Es empfiehlt sich daher, zunächst kleinere Experimente ohne FXML durchzuführen, um das Grundverständnis für die API zu entwickeln.
Ein weiterer Punkt, den es zu beachten gilt, ist die Integration von JavaFX in modernen Entwicklungsumgebungen. Seit Java 11 ist JavaFX kein Bestandteil mehr des JDK und muss als externe Bibliothek (z. B. über Maven oder Gradle) eingebunden werden. Dies kann in der Projektkonfiguration zusätzlichen Aufwand bedeuten.
Im Kontext unseres Spiels wird JavaFX verwendet, um Spielmenüs, Anzeigen und einfache grafische Rückmeldungen darzustellen. Die grafische Repräsentation der physikalischen Objekte erfolgt ebenfalls in JavaFX, während jBox2D ausschließlich für die Physiksimulation zuständig ist. Durch diese klare Trennung ist es möglich, Logik und Darstellung modular zu halten.

Fazit

Der Einsatz von JavaFX bietet uns eine moderne und flexible Plattform zur Entwicklung der Benutzeroberfläche. Erste Tests bestätigen, dass sich UI-Elemente einfach erstellen und dynamisch verändern lassen. In Kombination mit der Physiksimulation durch jBox2D ergibt sich eine leistungsfähige Architektur für die Umsetzung unseres Spiels.